

About the Prop AMM Competition

Designed by [Benedict Brady](#) and [Dan Robinson](#)

What's Different from Simple AMM?

In the Simple AMM competition, your strategy only controls **fees**. The swap itself always follows the constant-product formula ($x \cdot y = k$) — you just decide how much to charge on top. The market conditions are also relatively narrow: volatility barely varies, order flow is predictable, and the normalizer is a fixed 30bps pool.

Prop AMM changes both what you control and the environment you operate in. You control the **entire swap function** — given reserves and an input amount, your program decides exactly how much to output. This means you can implement any pricing curve you want: constant-product, concentrated liquidity, dynamic spreads, or something entirely new.

The market model is also significantly harder. There are **five randomized hyperparameters** instead of three, with much wider ranges — volatility spans 0.01%–0.70% per step (vs the Simple AMM's narrow 0.088%–0.101%), and two entirely new dimensions — normalizer fee and normalizer liquidity — vary across simulations. A strategy tuned to one market regime will fail in others.



Your strategy is written in Rust, compiled to BPF, and executed in a sandboxed environment.

The Challenge

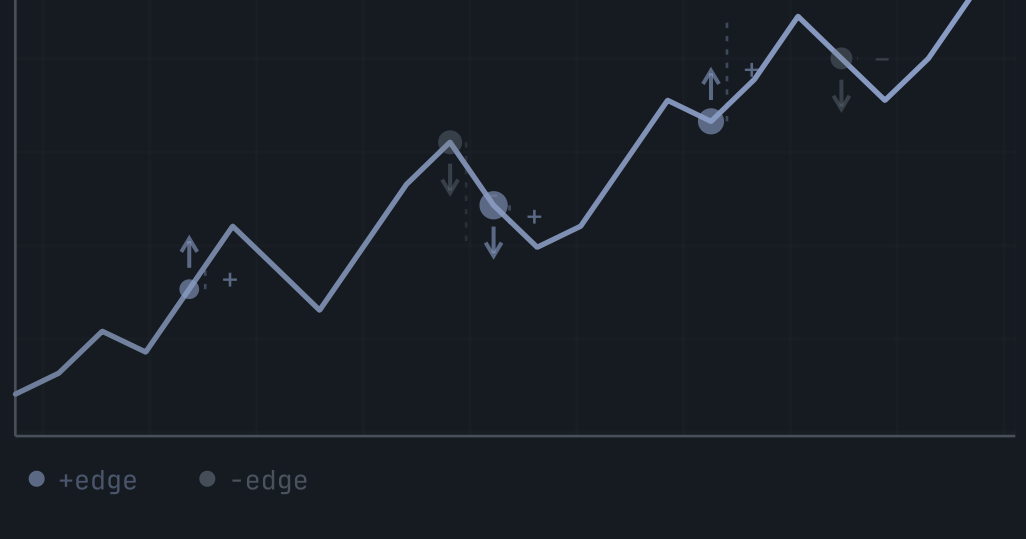
Your goal is to maximize **edge** — the profit your AMM earns relative to the true market price. You face both arbitrage bots (who extract value when your price is stale) and retail traders (who generate profit through the spread). With full control over pricing, you can be more aggressive or more conservative than a fixed curve would allow — but you also have more rope to hang yourself with.

How Scoring Works

Strategies are scored by **edge** — for each trade, how much you made or lost compared to the true price of the asset. If the AMM sells X above the true price or buys X below it, that's positive edge.

Edge =
$$\begin{aligned} &\sum (\text{amount_x} \times \text{true_price} - \text{amount_y}) && \text{for sells (AMM sells X)} \\ &+ \sum (\text{amount_y} - \text{amount_x} \times \text{true_price}) && \text{for buys (AMM buys X)} \end{aligned}$$

Retail trades generate positive edge (you profit from the spread). Arbitrage trades generate negative edge (informed flow extracts value). Your total edge is aggregated across 1000 randomized simulations with varying market conditions.



The Simulation

Each simulation runs 10,000 steps with a price process and retail order flow. Before each simulation, hyperparameters (volatility, arrival rate, order size, normalizer fee, and normalizer liquidity) are sampled randomly from a range — your strategy doesn't know these values and must adapt from observed trades.

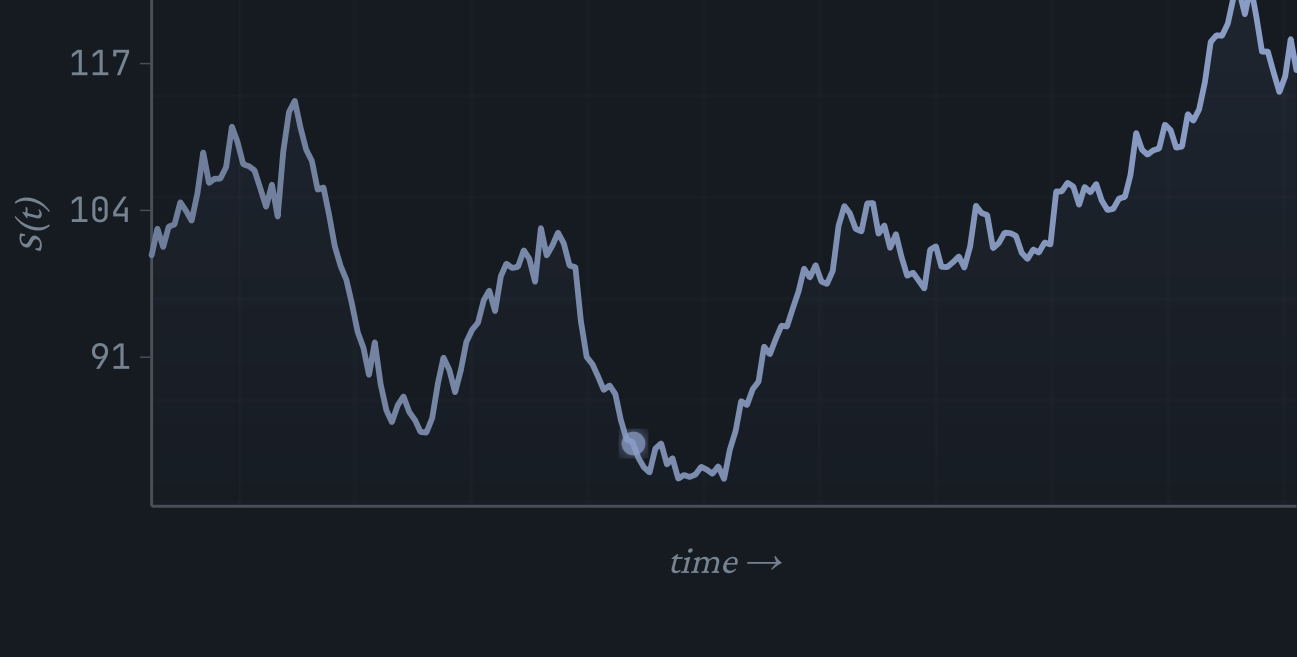
Price Process (GBM)

The true market price follows Geometric Brownian Motion:

$$S(t+1) = S(t) \cdot \exp(-\sigma^2/2 + \sigma Z)$$

where $Z \sim N(0,1)$

Drift $\mu = 0$ (no directional bias). Per-step volatility σ is sampled uniformly from [0.01%, 0.70%] at the start of each simulation and held fixed throughout.



Retail Order Flow

Uninformed traders arrive via Poisson process. Each simulation samples its own arrival rate and order size distribution:

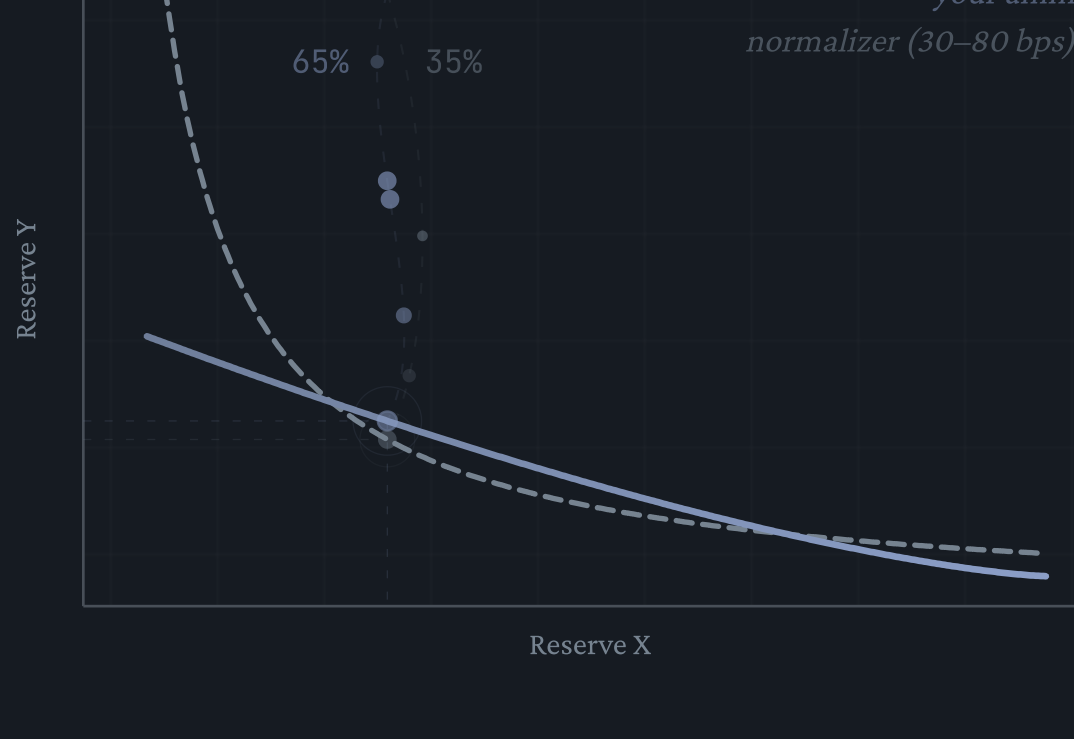
arrivals ~ Poisson(λ) $\lambda \sim U[0.4, 1.2]$ per sim
size ~ LogNormal($\mu, \sigma=1.2$) mean ~ $U[12, 28]$ in Y terms
direction = buy with prob 0.5, sell with prob 0.5

Normalizer AMM

The competing AMM is a constant-product market maker with randomized configuration each simulation: its fee ranges from 30 to 80 bps and its liquidity is scaled by a multiplier from 0.4x to 2.0x your reserves. A strategy tuned for one normalizer setup will underperform across the full distribution.

Order Routing

Orders are split optimally between your AMM and the normalizer. The router tries different split ratios and picks the one that maximizes total output for the trader. If your pricing is worse than the normalizer, flow routes away from you.



Why There's Another AMM

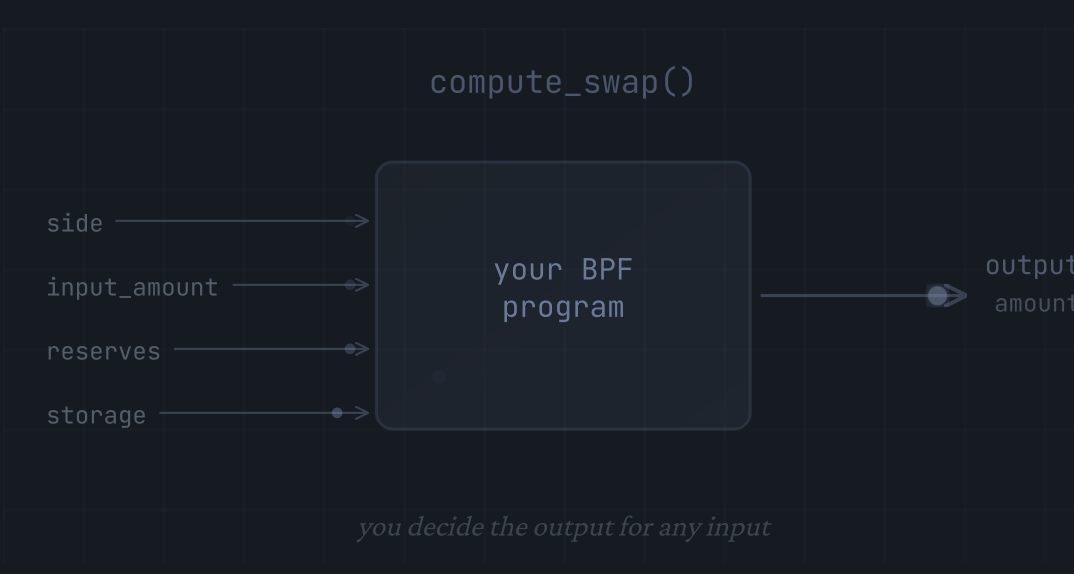
Your strategy shares the market with a competing constant-product AMM whose fee and liquidity vary each simulation. This **normalizer** prevents degenerate strategies from appearing profitable. If your prices are bad, retail flow routes to the normalizer and you earn nothing. The goal is to maximize your own edge, not to “beat” the other AMM.

Writing a Strategy

Your strategy is a Rust program compiled to BPF. It handles instructions via a single `process_instruction` entrypoint. There are two instruction types:

compute_swap (tag 0 or 1)

Called to price a trade. Given reserves and an input amount, return the output amount. This is called during both routing (to get quotes) and execution (to settle trades).



```
// Instruction data layout (25 bytes + 1024 bytes storage):
// [0]      side      0=buy X (Y→X), 1=sell X (X→Y)
// [1..9]    input_amount  u64 little-endian (1e9 scale)
// [9..17]   reserve_x   u64 little-endian (1e9 scale)
// [17..25]  reserve_y   u64 little-endian (1e9 scale)
// [25..]    storage     1024 bytes (read-only during swap)
//
// Return: 8-byte u64 output_amount via sol_set_return_data
```

afterSwap (tag 2)

Called after each real trade (not during routing quotes). Use this to update persistent state — track volumes, detect patterns, adjust parameters. You have 1024 bytes of mutable storage.

```
// Instruction data layout (34 bytes + 1024 bytes storage):
// [0]      tag=2
// [1]      side      0=buy, 1=sell
// [2..10]   input_amount  u64
// [10..18]  output_amount u64
// [18..26]  reserve_x   u64 (post-trade)
// [26..34]  reserve_y   u64 (post-trade)
// [34..]    storage     1024 bytes (read/write)
//
// Update storage via sol_set_storage syscall
```

Example: Constant-Product with 5% Fee

The starter program implements a simple constant-product curve with a static fee:

```
use pinocchio::{account_info::AccountInfo, entrypoint, pubkey::Pubkey, ProgramResult};
use prop_amm_submission_sdk::{set_return_data_bytes, set_return_data_u64};

const NAME: &str = "CFMM 5% Fee";
const MODEL_USED: &str = "None"; // Use "None" for human-written submissions.
const FEE_NUMERATOR: u128 = 950; // 95% passes through (5% fee)
const FEE_DENOMINATOR: u128 = 1000;
const STORAGE_SIZE: usize = 1024;

#[derive(wincode::SchemaRead)]
struct ComputeSwapInstruction {
    side: u8,
    input_amount: u64,
    reserve_x: u64,
    reserve_y: u64,
    _storage: [u8; STORAGE_SIZE],
}

#[cfg(not(feature = "no-entrypoint"))]
entrypoint!(process_instruction);

pub fn process_instruction(
    _program_id: &Pubkey,
    _accounts: &[AccountInfo],
    instruction_data: &[u8],
) -> ProgramResult {
    if instruction_data.is_empty() { return Ok(()); }

    match instruction_data[0] {
        0 | 1 => {
            let output = compute_swap(instruction_data);
            set_return_data_u64(output);
        }
        2 => { /* afterSwap - update storage here */ }
        3 => set_return_data_bytes(NAME.as_bytes()),
        4 => set_return_data_bytes(get_model_used().as_bytes()),
        _ => {}
    }
    Ok(())
}

pub fn get_model_used() -> &'static str {
    MODEL_USED
}

pub fn compute_swap(data: &[u8]) -> u64 {
    let decoded: ComputeSwapInstruction = match wincode::deserialize(data) {
        Ok(decoded) => decoded,
        Err(_) => return 0,
    };

    let input = decoded.input_amount as u128;
    let rx = decoded.reserve_x as u128;
    let ry = decoded.reserve_y as u128;

    if rx == 0 || ry == 0 { return 0; }
    let k = rx * ry;

    match decoded.side {
        0 => { // Buy X: Y in, X out
            let net = input * FEE_NUMERATOR / FEE_DENOMINATOR;
            let new_ry = ry + net;
            rx.saturating_sub((k + new_ry - 1) / new_ry) as u64
        }
        1 => { // Sell X: X in, Y out
            let net = input * FEE_NUMERATOR / FEE_DENOMINATOR;
            let new_rx = rx + net;
            ry.saturating_sub((k + new_rx - 1) / new_rx) as u64
        }
        _ => 0,
    }
}
```

But you're not limited to constant-product. You could implement concentrated liquidity, volatility-dependent spreads, asymmetric pricing, or any other approach — as long as your swap function is monotonic and convex.

Rules & Constraints

- **Monotonic:** larger input must produce larger output
- **Convex:** marginal price must worsen with trade size
- Compute budget: under 100,000 compute units per instruction
- Storage: 1024 bytes of persistent state across trades
- No external calls or system interactions
- All amounts use fixed-point scale (1e9)
- Code is compiled to BPF and executed in a sandboxed VM

Submit Your Strategy